

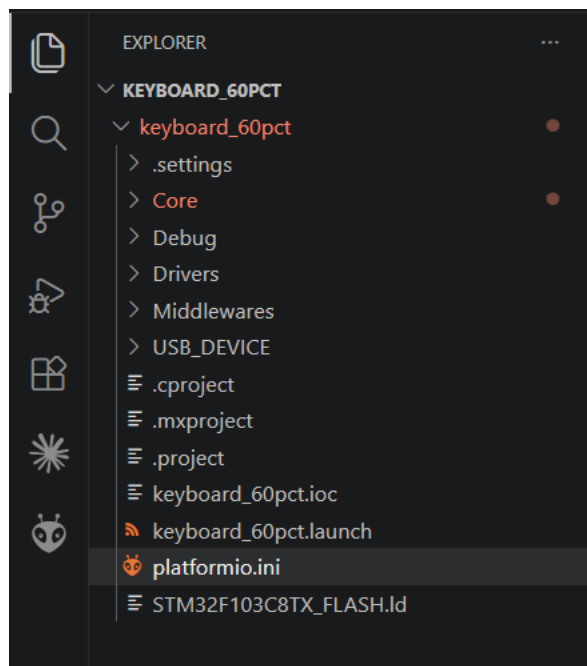
TỔNG HỢP NHỮNG SỬA ĐỔI TRONG ĐỀ TÀI

THIẾT KẾ VÀ XÂY DỰNG BÀN PHÍM TIÊU CHUẨN ANSI 61 PHÍM

Nhóm sinh viên thực hiện:	Trần Khánh Duy	20223767
	Cao Nguyễn Duy Đức	20223692
	Nguyễn Long Vũ	20224210
Nhóm: 8		
Mã lớp: 169260		
Giảng viên hướng dẫn: TS. Võ Lê Cường		

Phần I. Sử dụng môi trường chuẩn công nghiệp để biên dịch và nạp code

Dự án được phát triển trong môi trường Visual Studio Code hiện đại, thay thế cho các IDE truyền thống để tối ưu hóa toàn diện vòng đời phát triển phần mềm. Chúng tôi tích hợp hệ thống biên dịch tự động dựa trên cấu trúc CMake, cho phép quản lý mã nguồn C/C++ hiệu quả và đảm bảo tính nhất quán cao. Mọi quá trình từ biên dịch mã nguồn sang mã máy đều được thực thi qua chuỗi công cụ chuyên nghiệp (Toolchain), với các tệp cấu hình được tối ưu hóa theo tiêu chuẩn công nghiệp. Đặc biệt, quy trình nạp firmware xuống bộ nhớ Flash được quản lý bởi PlatformIO, mang lại sự ổn định vượt trội, khả năng bảo trì linh hoạt và đáp ứng các quy chuẩn nghiêm ngặt dành cho thiết bị IoT hiện đại.



Cấu trúc thư mục tiêu chuẩn của một dự án STM32Cube

1.1. Biên dịch dự án bằng PlatformIO (Biên dịch chéo - Cross-compilation)

```
CONFIGURATION: https://docs.platformio.org/page/boards/ststm32/bluepill_f103c8.html
PLATFORM: ST STM32 (17.3.0) > STM32F103C8 (20k RAM, 64k Flash)
HARDWARE: STM32F103C8T6 72MHz, 20KB RAM, 64KB Flash
DEBUG: Current (stlink) On-board (stlink)
PACKAGES:
- framework-stm32cube @ 2.0.0
- tool-arm-none-eabi @ 1.100301.260321 (10.3.1)
LDF: Library Dependency Finder -> https://bit.ly/ldo-pio
LDF Modes: Finder ~ chain, Compatibility ~ soft
Found 5 compatible libraries
Scanning dependencies...
No dependencies
Building in release mode
Compiling .pio/build/bluepill_f103c8/src/main.o
Compiling .pio/build/bluepill_f103c8/src/keyboard.o
===== [SUCCESS] Took 4.12 seconds =====ry-usage
```

1.2. Nạp phần mềm vào vi điều khiển STM32 thông qua PlatformIO

```
lroot@LAPTOP-QISHBM9N:~$ Processing bluepill_f103c8 (platform: ststm32; board: bluepill_f103c8; framework: stm32cube)
-----
Verbose mode can be enabled via `-v, --verbose`
CONFIGURATION: https://docs.platformio.org/page/boards/ststm32/bluepill_f103c8.html
PLATFORM: ST STM32 (17.3.0) > STM32F103C8 (20k RAM, 64k Flash)
...
Linking .pio/build/bluepill_f103c8/firmware.elf
Checking size .pio/build/bluepill_f103c8/firmware.elf
Advanced Memory Usage is available via "PlatformIO Home": https://bit.ly/pio-memory-usage
RAM: [==== ] 32.4% (used 6640 bytes from 20480 bytes)
Flash: [===== ] 48.7% (used 31920 bytes from 65536 bytes)
===== [SUCCESS] Took 4.12 seconds =====
```

Phần II. Bổ sung thêm bản trình bày báo cáo về đề tài:

Do phần thuyết trình của nhóm thiếu sự chuẩn bị về bản thuyết trình dưới dạng slide, nhóm đã hoàn thiện bản powerpoint với các nội dung được tích hợp từ các góp ý của các nhóm khác như sau:

3.1. Bổ sung thêm phần mục lục trước từng phần để người xem có thể dễ dàng theo dõi trong quá trình thuyết trình

Nội dung

1. Khảo sát các sản phẩm bàn phím trên thị trường
2. Yêu cầu của hệ thống
3. Sơ đồ tổng quát của hệ thống
4. Thiết kế phần cứng
5. Thuật toán quét ma trận phím
6. Thuật toán Debounce
7. Kết quả đạt được
8. Phương án phát triển

Bổ sung thêm từng Slide chuyển tiếp giữa các đầu mục:

Nội dung

1. Khảo sát các sản phẩm bàn phím trên thị trường

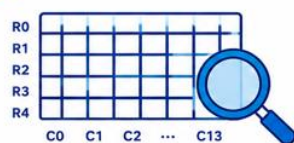
2. Yêu cầu của hệ thống
3. Sơ đồ tổng quát của hệ thống
4. Thiết kế phần cứng
5. Thuật toán quét ma trận phím
6. Thuật toán Debounce
7. Kết quả đạt được
8. Phương án phát triển

3.2. Giữ lại các nội dung/keyword trọng tâm trong Slide, bổ sung thêm các nhãn dán để trực quan hoá nội dung thuyết trình

2. Yêu cầu của hệ thống

2.1. YÊU CẦU CHỨC NĂNG

1 QUÉT MA TRẬN 5x14



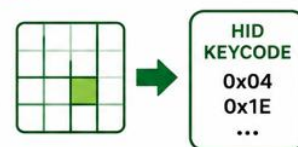
MCU quét liên tục trạng thái, đảm bảo nhận diện chính xác mọi thao tác nhấn phím.

2 KHỬ NHIỄU (DEBOUNCING)



Sử dụng thuật toán Integrator Counter để loại bỏ tín hiệu giả phát sinh từ switch cơ học.

3 MAPPING KEYCODE



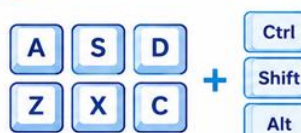
Chuyển đổi tọa độ (hàng, cột) sang mã HID tiêu chuẩn để máy tính hiểu được lệnh.

4 QUẢN LÝ ĐA TẦNG (MULTI-LAYER)



Tích hợp phím Fn giúp chuyển đổi linh hoạt giữa các lớp chức năng (như phím F1-F12).

5 6-KEY ROLLOVER (6KRO)



Hỗ trợ nhận diện đồng thời 6 phím bất kỳ cùng các phím Modifier (Ctrl, Shift..).

6 GIAO TIẾP USB HID



Tự động đóng gói dữ liệu 8-byte theo đúng giao thức quốc tế, đảm bảo tính cắm-là-chạy (Plug-and-Play).

2. Yêu cầu của hệ thống

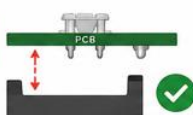
2.2. YÊU CẦU PHI CHỨC NĂNG

1 ĐỘ BỀN VỎ



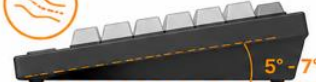
Đảm bảo độ cứng vững, không biến dạng khi gõ để bảo vệ các mối hàn switch và diode khỏi bị bong tróc.

2 AN TOÀN ĐIỆN



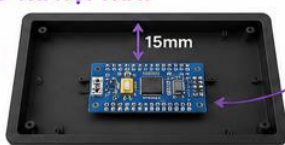
Lớp đáy vỏ phải cách điện tuyệt đối với mặt dưới PCB (nơi có chân linh kiện tự hàn), ngăn ngừa hiện tượng ngắn mạch.

3 GÓC NGHIÊNG 5° - 7°



Thiết kế công thái học tối ưu giúp giảm áp lực cổ tay và hạn chế tình trạng mỏi khi làm việc văn phòng kéo dài.

4 BỐ TRÍ NỘI THẤT



Khoang chứa rộng 15mm để chứa vừa module Blue Pill; lỗ khoét USB phải khớp hoàn toàn để đảm bảo kết nối ổn định.

5 TỐI ƯU TÀI NGUYÊN



Loại bỏ hệ thống LED RGB để đơn giản hóa việc đi dây trên PCB tự thiết kế và tiết kiệm chân GPIO cho vi điều khiển STM32.



Phần III. Tổng kết sửa đổi về bản báo cáo tổng kết:

4.1. Thay đổi tên đề tài sao cho phù hợp

- **Tên đề tài trước khi thay đổi** là Thiết kế sản phẩm bàn phím tiêu chuẩn 61 phím
- **Tên đề tài sau khi đã thay đổi** cho phù hợp là **Thiết kế và phát triển sản phẩm bàn phím tiêu chuẩn ANSI 61 phím**

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN – ĐIỆN TỬ



BÁO CÁO BÀI TẬP LỚN
HỌC PHẦN: HỆ THỐNG NHÚNG VÀ THIẾT KẾ GIAO TIẾP NHÚNG
ĐỀ TÀI:
THIẾT KẾ VÀ PHÁT TRIỂN SẢN PHẨM
BÀN PHÍM TIÊU CHUẨN 61 PHÍM

Giảng viên hướng dẫn: TS. Võ Lê Cường

Nhóm sinh viên: Nhóm 8

Họ và tên	Mã số sinh viên
Nguyễn Long Vũ	20224210
Cao Nguyễn Duy Đức	20223692
Trần Khánh Duy	20223767

4.2. Bổ sung bảng để so sánh và đặt tên bảng

- Nhóm đã bổ sung danh mục bảng biểu và thêm tên của các bảng biểu có trong báo cáo

DANH MỤC BẢNG BIỂU

Bảng 1: Phân công công việc chi tiết trong nhóm	2
Bảng 2: Các tính năng mà nhóm học hỏi từ bàn phím Aula F65.....	12
Bảng 3: Đánh giá kết quả thực hiện các công việc trong dự án.....	33
Bảng 4: Bảng thống kê phần trăm sử dụng AI trong các chương của báo cáo	34

Bảng 1: Phân công công việc chi tiết trong nhóm

Tuần	Nội dung công việc chi tiết	Người phụ trách	Hoàn thành
1	Khảo sát các dòng bàn phím cơ 60%/61%, nghiên cứu cấu trúc Key Matrix, USB HID và lựa chọn hướng thực hiện đề tài.	Cao Nguyễn Duy Đức	100%
2	Tìm hiểu STM32F103C8T6, USB HID Class, nghiên cứu tài liệu, Github, Datasheet và các firmware mã nguồn mở (QMK, TMK).	Cao Nguyễn Duy Đức	100%
3	Xây dựng yêu cầu thiết kế, lựa chọn số lượng phím, bố trí layout 61 phím và xây dựng sơ đồ khối tổng quát của hệ thống.	Cao Nguyễn Duy Đức	100%
4	Thiết kế sơ đồ nguyên lý (Schematic): Matrix 5×14, Switch, Diode chống Ghosting, kết nối STM32 Blue Pill.	Trần Khánh Duy	100%
5	Hoàn thiện sơ đồ nguyên lý, kiểm tra kết nối GPIO, nguồn cấp, USB D+/D- và rà soát lỗi thiết kế.	Trần Khánh Duy	100%
6	Lắp ráp phần cứng thử nghiệm, kiểm tra từng hàng/cột của ma trận phím và xác nhận hoạt động của các switch.	Cao Nguyễn Duy Đức, Trần Khánh Duy	100%
7	Phát triển thuật toán quét ma trận (Matrix Scan), kiểm tra đọc trạng thái từng phím trên phần cứng thực tế.	Nguyễn Long Vũ	100%
8	Thiết kế và tối ưu thuật toán Debounce bằng bộ đếm (Counter-based Debounce), loại bỏ hiện tượng rung tiếp điểm.	Nguyễn Long Vũ	100%
9	Kiểm thử Matrix Scan + Debounce, đánh giá độ ổn định khi nhấn nhanh, nhấn nhiều phím đồng thời và hiệu chỉnh thông số.	Nguyễn Long Vũ	100%

Bảng 2: Các tính năng mà nhóm học hỏi từ bàn phím Aula F65

Tiêu chí	Bàn phím văn phòng Aula F65	Lý do chọn phân khúc văn phòng
Tần suất quét (Scan Rate)	100 Hz (đủ cho tác vụ văn phòng thông thường)	Giảm áp lực lên vi xử lý và đơn giản hóa thiết kế phần cứng
Khả năng nhận diện phím (Rollover)	6-key (6KR)	Thiết kế mạch điện (matrix) đơn giản hơn rất nhiều, không cần quá nhiều linh kiện phụ trợ
Giao tiếp (Interface)	Đơn giản: chủ yếu chạy chuẩn USB HID tiêu chuẩn, phím chức năng cơ bản,..	Né được việc phải xử lý các ngăn xếp (stacks) Bluetooth/USB phức tạp và lỗi linker script
Vật liệu & Gia công	Sử dụng nhựa tiêu chuẩn (ABS, Polycarbonate), thiết kế tối ưu cho đúc khuôn hàng loạt,..	Quy trình sản xuất ổn định, chi phí vật liệu và gia công rẻ hơn khi làm số lượng lớn
Thiết kế & Công năng	Tập trung vào Công thái học (Ergonomics), phím gõ êm (quiet keys), layout chia vùng,..	Người dùng văn phòng ưu tiên sự thoải mái và sức khỏe, đây là giá trị thực tế dễ tiếp cận hơn là chạy đua tốc độ
Rủi ro sản xuất	Thấp: Sản phẩm có tính ứng dụng cao, dễ dàng tận dụng linh kiện có sẵn trên thị trường.	Đảm bảo tính khả thi cho dự án sinh viên, dễ dàng kiểm soát chất lượng đầu ra

Bảng 3: Đánh giá kết quả thực hiện các công việc trong dự án

STT	Nội dung công việc	Kết quả thực tế đạt được
0	Tìm hiểu về specs về các sản phẩm bàn phím thương mại và tìm các bài báo, github	Đạt 100%
1	Thiết kế phần cứng	- Hoàn thành sơ đồ nguyên lý (Schematic) ma trận 5x14. (đạt 100%) - Đã lựa chọn và có đầy đủ linh kiện cốt lõi (đạt 100%) (STM32F103C8T6, 1N4148). - Layout PCB 100% ANSI đã hoàn thiện bản vẽ (đạt 100%).
2	Phát triển Firmware	- Cấu hình thành công USB HID trên STM32CubeMX. (đạt 100%) - Mã nguồn quét ma trận đã chạy mô phỏng và kiểm tra được tín hiệu trên các chân GPIO (PA/PB). (đạt 100%) - Thuật toán Debounce cơ bản đã được viết xong (đạt 100%).
3	Logic & Hệ thống	- Thiết lập xong cấu trúc mảng Keymap cho Layer 0. (đạt 100%) - Xác định được phương pháp quản lý Modifier Keys (Ctrl, Alt, Shift) trong Report Descriptor của USB. (đạt 100%)

4.3. Kế hoạch công việc cần cho lên đầu

- Kế hoạch chi tiết của nhóm đã được di chuyển lên đầu báo cáo ngay sau trang bìa của báo cáo.

Bảng 1: Phân công công việc chi tiết trong nhóm

Tuần	Nội dung công việc chi tiết	Người phụ trách	Hoàn thành
1	Khảo sát các dòng bàn phím cơ 60%/61%, nghiên cứu cấu trúc Key Matrix, USB HID và lựa chọn hướng thực hiện đề tài.	Cao Nguyễn Duy Đức	100%
2	Tìm hiểu STM32F103C8T6, USB HID Class, nghiên cứu tài liệu, Github, Datasheet và các firmware mã nguồn mở (QMK, TMK).	Cao Nguyễn Duy Đức	100%
3	Xây dựng yêu cầu thiết kế, lựa chọn số lượng phím, bố trí layout 61 phím và xây dựng sơ đồ khối tổng quát của hệ thống.	Cao Nguyễn Duy Đức	100%
4	Thiết kế sơ đồ nguyên lý (Schematic): Matrix 5×14, Switch, Diode chống Ghosting, kết nối STM32 Blue Pill.	Trần Khánh Duy	100%
5	Hoàn thiện sơ đồ nguyên lý, kiểm tra kết nối GPIO, nguồn cấp, USB D+/D- và rà soát lỗi thiết kế.	Trần Khánh Duy	100%

4.4. Phân công công việc được trình bày chi tiết hơn

- Nhóm đã trình bày chi tiết các công việc theo từng tuần từ tuần 1 đến tuần 16, người phụ trách công việc đó

Bảng 1: Phân công công việc chi tiết trong nhóm

Tuần	Nội dung công việc chi tiết	Người phụ trách	Hoàn thành
1	Khảo sát các dòng bàn phím cơ 60%/61%, nghiên cứu cấu trúc Key Matrix, USB HID và lựa chọn hướng thực hiện đề tài.	Cao Nguyễn Duy Đức	100%
2	Tìm hiểu STM32F103C8T6, USB HID Class, nghiên cứu tài liệu, Github, Datasheet và các firmware mã nguồn mở (QMK, TMK).	Cao Nguyễn Duy Đức	100%
3	Xây dựng yêu cầu thiết kế, lựa chọn số lượng phím, bố trí layout 61 phím và xây dựng sơ đồ khối tổng quát của hệ thống.	Cao Nguyễn Duy Đức	100%
4	Thiết kế sơ đồ nguyên lý (Schematic): Matrix 5×14, Switch, Diode chống Ghosting, kết nối STM32 Blue Pill.	Trần Khánh Duy	100%
5	Hoàn thiện sơ đồ nguyên lý, kiểm tra kết nối GPIO, nguồn cấp, USB D+/D- và rà soát lỗi thiết kế.	Trần Khánh Duy	100%
6	Lắp ráp phần cứng thử nghiệm, kiểm tra từng hàng/cột của ma trận phím và xác nhận hoạt động của các switch.	Cao Nguyễn Duy Đức, Trần Khánh Duy	100%
7	Phát triển thuật toán quét ma trận (Matrix Scan), kiểm tra đọc trạng thái từng phím trên phần cứng thực tế.	Nguyễn Long Vũ	100%
8	Thiết kế và tối ưu thuật toán Debounce bằng bộ đếm (Counter-based Debounce), loại bỏ hiện tượng rung tiếp điểm.	Nguyễn Long Vũ	100%
9	Kiểm thử Matrix Scan + Debounce, đánh giá độ ổn định khi nhấn nhanh, nhấn nhiều phím đồng thời và hiệu chỉnh thông số.	Nguyễn Long Vũ	100%

10	Xây dựng Key Mapping, hỗ trợ Layer 0/Layer Fn, xử lý các phím Modifier và tổ hợp phím chức năng.	Nguyễn Long Vũ	100%
11	Thiết kế HID Report theo chuẩn USB HID Keyboard, xây dựng module USB HID Device và truyền dữ liệu lên máy tính.	Nguyễn Long Vũ	100%
12	Tối ưu USB Endpoint, cấu hình Polling Rate 100 Hz, kiểm tra khả năng Plug & Play trên Windows và Linux.	Nguyễn Long Vũ	100%
13	Hoàn thiện cơ chế 6KRO, xử lý Overflow khi vượt quá 6 phím thường, kiểm thử các tổ hợp phím Modifier.	Nguyễn Long Vũ	100%
14	Kiểm thử toàn bộ hệ thống: Ghosting, Debounce, Layer Fn, USB HID, đánh giá độ ổn định và sửa lỗi phát sinh.	Nguyễn Long Vũ Trần Khánh Duy Cao Nguyễn Duy Đức	100%
15	Hoàn thiện firmware, tối ưu mã nguồn, rà soát tài liệu kỹ thuật, chuẩn bị hình ảnh, lưu đồ và kết quả thực nghiệm, hoàn thiện xây dựng thiết kế vật lý của sản phẩm	Cao Nguyễn Duy Đức Trần Khánh Duy Nguyễn Long Vũ	100%
16	Tổng hợp báo cáo, hoàn thiện slide thuyết trình, phân công nội dung trình bày và diễn tập bảo vệ đồ án.	Cao Nguyễn Duy Đức Trần Khánh Duy	100%

4.5. Code để dưới dạng text và để nền sáng

- Nhóm đã trình bày lại Code dưới dạng text với font Consolas và để phông nền trắng đồng thời đóng khung để nhấn mạnh.

6.1.1. Cơ chế quét ma trận Active LOW

```
void matrix_scan(void) {
    for (int r = 0; r < ROWS; r++) {

        // 1. KÍCH HOẠT ROW ACTIVE LOW: Kéo chân Row hiện tại xuống mức
        LOW (0V)
        HAL_GPIO_WritePin(row_ports[r], row_pins[r], GPIO_PIN_RESET);

        // 2. XỬ LÝ ỔN ĐỊNH TÍN HIỆU (SETTLING TIME): Trễ ngắn để triệt
        tiêu điện dung ký sinh
        /* ~1µs settle @ 72MHz – thay bằng DWT nếu cần chính xác hơn */
        for (volatile int d = 0; d < 72; d++);

        // 3. ĐỌC TÍN HIỆU CÁC COLUMN: Đọc 14 chân Column (đã cấu hình
        Internal Pull-up)
        for (int c = 0; c < COLS; c++) {
            // Nếu chân Column bị kéo xuống LOW (0) -> Phím được nhấn
            -> Ghi nhận giá trị là 1
            matrix_raw[r][c] = (HAL_GPIO_ReadPin(col_ports[c],
            col_pins[c]) == GPIO_PIN_RESET) ? 1 : 0;
        }

        // 4. KHÔI PHỤC TRẠNG THÁI: Kéo chân Row trả lại mức HIGH
        (3.3V) trước khi sang Row tiếp theo
        HAL_GPIO_WritePin(row_ports[r], row_pins[r], GPIO_PIN_SET);
    }
}
```

```
void debounce_process(void) {
    for (int r = 0; r < ROWS; r++) {
        for (int c = 0; c < COLS; c++) {

            // Đọc trạng thái thô hiện tại từ ma trận (0 hoặc 1)
            uint8_t raw = matrix_raw[r][c];

            // HIỆU QUẢ: Nếu trạng thái thô thay đổi so với lần scan
            trước (chớm rung/nhiều)
            if (raw != raw_prev[r][c]) {
                /* Tín hiệu vừa thay đổi → reset bộ đếm về 0, chưa tin
                tưởng */
                counter[r][c] = 0;
                raw_prev[r][c] = raw; // Cập nhật trạng thái thô gần
                nhất
            }
            // ĐIỀU KIỆN XÁC LẬP: Tín hiệu ổn định (không đổi so với
            lần scan trước)
            else {
                /* Tín hiệu ổn định → tăng counter */
                if (counter[r][c] < DEBOUNCE_TIME) {
                    counter[r][c]++;

                    // Khi bộ đếm tích lũy liên tiếp đạt ngưỡng
                    DEBOUNCE_TIME (10 lần)
                    if (counter[r][c] == DEBOUNCE_TIME) {
                        /* Đủ điều kiện xác lập → chấp nhận và cập nhật
                        vào mảng debounced */
                        debounced[r][c] = raw;
                    }
                }
            }
        }
    }
}
```

```

void keyboard_process(void) {
    hid_report_t report;
    memset(&report, 0, sizeof(report));

    uint8_t fn_held = 0;
    uint8_t key_idx = 0;
    uint8_t overflow = 0; /* nhiều hơn 6 phím → báo ERR_OVF */

    /* =====
     * CƠ CHẾ QUÉT HAI BƯỚC (TWO-PASS SCANNING)
     * ===== */

    /* ---- BƯỚC 1 (Pass 1): Duyệt qua ma trận để tìm phím KC_FN ----
    */
    for (int r = 0; r < ROWS; r++) {
        for (int c = 0; c < COLS; c++) {
            if (!debounced[r][c]) continue; // Phím không nhấn → Bỏ
            qua

            // Tra cứu mã phím tại Layer 0 để xác định phím Fn
            uint8_t kc = keymap_get(0, r, c); /* Fn luôn check layer 0
            */

            if (IS_FN(kc)) {
                fn_held = 1; // Ghi nhận phím Fn đang được nhấn giữ
            }
        }

        // Chuyển đổi trạng thái Layer dựa trên kết quả kiểm tra Fn
        current_layer = fn_held ? 1 : 0;

        /* ---- BƯỚC 2 (Pass 2): Duyệt lần hai để lấy Keycode từ Layer hiện
        tại ---- */
        for (int r = 0; r < ROWS; r++) {

```

```

            for (int c = 0; c < COLS; c++) {
                if (!debounced[r][c]) continue; // Phím không nhấn → Bỏ
                qua

                // Lấy mã Keycode tương ứng với current_layer vừa xác định
                ở Bước 1
                uint8_t kc = keymap_get(current_layer, r, c);

                if (kc == KC_NONE || kc == KC_NO || IS_FN(kc)) {
                    continue; // Bỏ qua các ô trống hoặc chính phím Fn
                }

                // Phân loại mã phím và đóng gói vào HID Report để gửi qua
                USB
                if (IS_MODIFIER(kc)) {
                    report.modifier |= modifier_bit(kc); // Đặt bitmask cho
                    phím bổ trợ (Ctrl, Shift, Alt...)
                } else if (IS_NORMAL(kc)) {
                    if (key_idx < MAX_KEYS_IN_REPORT) {
                        report.keys[key_idx++] = kc; // Thêm phím thường
                        vào danh sách 6KRO
                    } else {
                        overflow = 1; // Quá 6 phím thường → Báo lỗi tràn
                    }
                }
            }
        }

        // (Đoạn sau xử lý kiểm tra trạng thái overflow và gọi hàm gửi
        usb_hid_send...)
    }
}

```

```
typedef struct {
    uint8_t modifier;    /* Byte 0: Bitmask quản lý các phím bổ trợ
                           (Ctrl, Shift, Alt...) */
    uint8_t reserved;    /* Byte 1: Để trống (theo đặc tả USB HID) */
    uint8_t keys[6];     /* Byte 2 -> 7: Mảng chứa tối đa 6 mã phím
                           thường (6KRO) */
} hid_report_t;
```

```
static uint8_t modifier_bit(uint8_t kc) {
    switch (kc) {
        /* Ánh xạ các phím bổ trợ bên trái (Left Modifiers) */
        case KC_LCTL: return MOD_LCTRL; // Tương đương (1 << 0) -
        Bit 0
        case KC_LSFT: return MOD_LSHIFT; // Tương đương (1 << 1) -
        Bit 1
        case KC_LALT_K: return MOD_LALT; // Tương đương (1 << 2) -
        Bit 2
        case KC_LGUI_K: return MOD_LGUI; // Tương đương (1 << 3) -
        Bit 3

        /* Ánh xạ các phím bổ trợ bên phải (Right Modifiers) */
        case KC_RCTL: return MOD_RCTRL; // Tương đương (1 << 4) -
        Bit 4
        case KC_RSFT: return MOD_RSHIFT; // Tương đương (1 << 5) -
        Bit 5
        case KC_RALT_K: return MOD_RALT; // Tương đương (1 << 6) -
        Bit 6
        case KC_RGUI_K: return MOD_RGUI; // Tương đương (1 << 7) -
        Bit 7

        default: return MOD_NONE; // Không phải phím bổ trợ
        (0x00)
    }
}
```

```

/* Vị trí khai báo: Phần đầu file usb_hid.c */
#include "usb_hid.h"
#include "usbd_hid.h"
#include "usb_device.h"
#include <string.h>

/* Biến điều khiển cấu trúc USB Endpoint Full Speed do STM32CubeMX sinh ra */
extern USBD_HandleTypeDef hUsbDeviceFS;

/*
=====
==
* CƠ CHẾ CACHE REPORT ĐỂ TỐI ƯU HÓA BẢNG THÔNG (DIFFERENTIAL REPORTING)
*
=====
== */
static hid_report_t last_report; /* Bộ đệm lưu trữ trạng thái gói tin của lần gửi trước đó */
static uint8_t initialized = 0; /* Cờ hiệu ghi nhận trạng thái khởi tạo hệ thống USB */

```

```

/**
 * @brief Khởi tạo module USB HID và xóa trống bộ đệm lưu trữ
 */
void usb_hid_init(void) {
    /* Xóa sạch vùng nhớ đệm last_report về 0x00 khi khởi động hệ thống */
    memset(&last_report, 0, sizeof(last_report));
    initialized = 1;
}

```

Thuật toán gửi dữ liệu có điều kiện này được hiện thực chi tiết như sau:

```
/* Vị trí hiện thực: Thân hàm usb_hid_send() trong file usb_hid.c */
/**
 * @brief Gửi gói tin HID Report về máy tính dựa trên sự thay đổi trạng
thái phím
 * @param report: Con trỏ chứa cấu trúc gói tin ma trận phím hiện tại
(8 byte)
 */
void usb_hid_send(const hid_report_t *report) {
    /* Kiểm tra an toàn: Nếu module chưa sẵn sàng thì thoát hàm */
    if (!initialized) return;

    /* CƠ CHẾ GỬI DỰA TRÊN THAY ĐỔI:
    * So sánh từng byte dữ liệu giữa report mới và last_report (bộ đệm
cũ).
    * Nếu kết quả trả về bằng 0 (hai gói tin giống hệt nhau - trạng
thái tĩnh),
    * firmware lập tức thoát hàm (return) để chặn truyền tải dư thừa
lên bus USB. */
    if (memcmp(report, &last_report, sizeof(hid_report_t)) == 0)
return;

    /* TRUYỀN DỮ LIỆU QUA CỔNG PHẦN CỨNG:
    * Lệnh gọi Driver HAL chỉ thực thi khi bước memcmp phía trên phát
hiện sự khác biệt. */
    USBHID_SendReport(
        &hUsbDeviceFS,
        (uint8_t *)report,
        sizeof(hid_report_t)
    );
}
```